

# SOFTWARE ARCHITECTURE DOCUMENT

## Table of Contents

1. Introduction
2. Architecture
3. Use Case View
4. Logical View
5. Sequence View
6. Quality

## 1. Introduction :

### Purpose :

The purpose of this project is to address a key limitation of traditional hash tables — their inability to handle an unknown number of items — by implementing a **dynamic hash table** that grows and shrinks incrementally. Although hash tables do not support range queries, they provide higher throughput for point queries than B-trees, making them a strong candidate for database index structures when only point queries are needed.

The project studies variants of dynamic hashing, selects **Linear Hashing** based on Larson (1988) and Litwin (1980), implements it with **parallel basic operations** in C++17, and benchmarks it against other data structures.

## Scope

- Study and comparison of dynamic hash table variants — Linear Hashing, Spiral Storage, Extendible Hashing
- Implementation of concurrent Linear Hashing in C++17 using Larson's two-level architecture
- Dynamic growth (split) and shrinking (merge) based on load factor
- Correctness testing and multi-threaded stress testing

## 2. Architectural Representation :

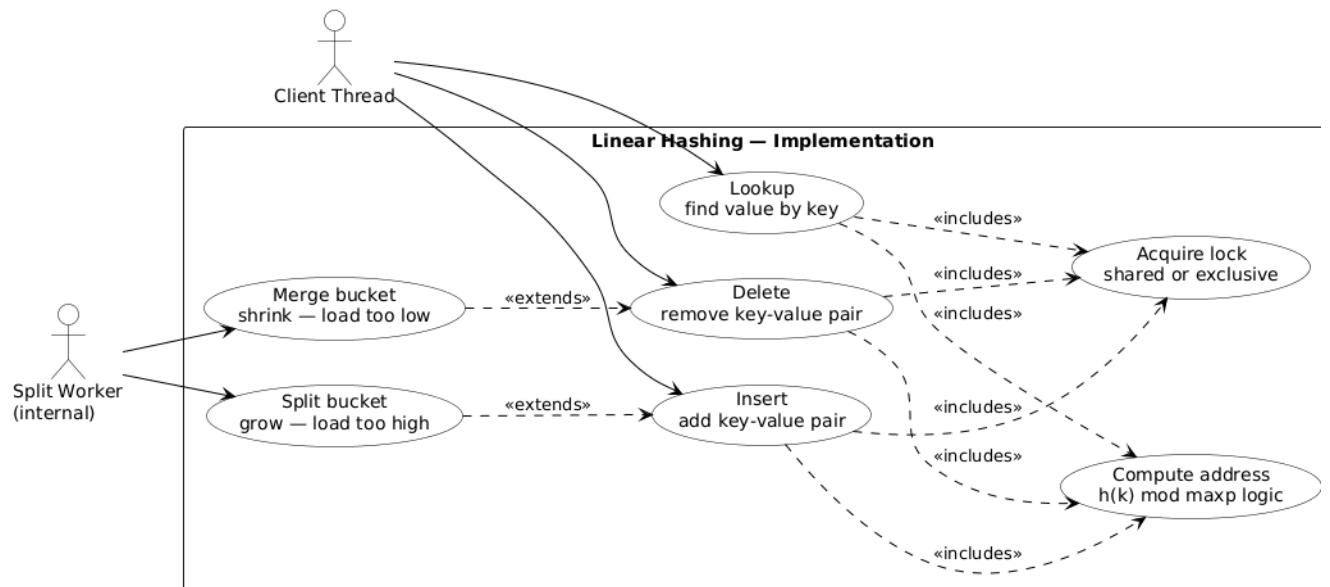
This document presents the architecture as a series of views; use case view, logical view, process view and deployment view.

# SOFTWARE ARCHITECTURE DOCUMENT

## Use Case View :

- Insert
- Lookup
- Delete
- Compute address
- Acquire lock
- Split bucket
- Merge bucket

### Architecturally-Significant Use Cases:



### Client Thread :

The actor that calls the hash table operations. Represents any application thread running insert, lookup, or delete simultaneously with other threads.

### Split Worker (internal) :

Not called directly by the user. Triggered automatically inside the system when the load factor crosses a threshold. Manages the growth and shrinking of the table.

# SOFTWARE ARCHITECTURE DOCUMENT

## **Insert — add key-value pair:**

A thread provides a key and a value. The system computes which bucket the key belongs to, locks that bucket exclusively, and appends the record to the bucket's linked list. If the load factor exceeds the upper threshold after the insert, a split is triggered.

## **Lookup — find value by key:**

A thread provides a key. The system computes the bucket address, acquires a shared lock (so other readers are not blocked), walks the linked list in that bucket, and returns the value if found or "not found" if the key does not exist.

## **Delete — remove key-value pair:**

A thread provides a key. The system computes the bucket address, locks the bucket exclusively, finds the matching record in the linked list, and removes it. If the load factor drops below the lower threshold after the delete, a merge is triggered.

## **Compute address — $h(k) \bmod \text{maxp}$ logic:**

Runs automatically inside every insert, lookup, and delete. Applies Larson's two-step formula: first compute  $\text{addr} = \text{hash}(\text{key}) \bmod (\text{maxp} + 1)$ . If  $\text{addr}$  is less than the split pointer  $p$ , recompute using  $\text{hash}(\text{key}) \bmod 2 * (\text{maxp} + 1)$ . The result is the exact bucket index where the key lives.

## **Split bucket — grow, load too high:**

When the load factor  $\alpha = \text{keycount} / (\text{maxp} + 1)$  exceeds the upper threshold, the split worker locks bucket  $p$  and creates a new bucket at position  $p + \text{maxp} + 1$ . Records in bucket  $p$  are redistributed between the two buckets using the updated hash function. The split pointer  $p$  is then incremented atomically.

## **Merge bucket — shrink, load too low:**

When the load factor drops below the lower threshold, the split worker reverses the last split. The records from the last bucket are moved back into its partner bucket. The last bucket is removed and the split pointer  $p$  is decremented atomically.

## **Acquire lock — shared or exclusive:**

## SOFTWARE ARCHITECTURE DOCUMENT

Runs automatically inside every operation before touching a bucket. Lookup acquires a shared lock — multiple threads can read the same bucket at the same time. Insert, delete, split, and merge acquire an exclusive lock — only one thread can write to a bucket at a time.

Implemented using `std::shared_mutex` per bucket.

### Logical View:

The logical view describes the internal structure of the Concurrent Linear Hash Table system. It identifies the key classes, their responsibilities, their attributes and methods, and the relationships between them. While the use case view describes *what* the system does from the outside, the logical view describes *how* the system is organised on the inside to deliver that behaviour.

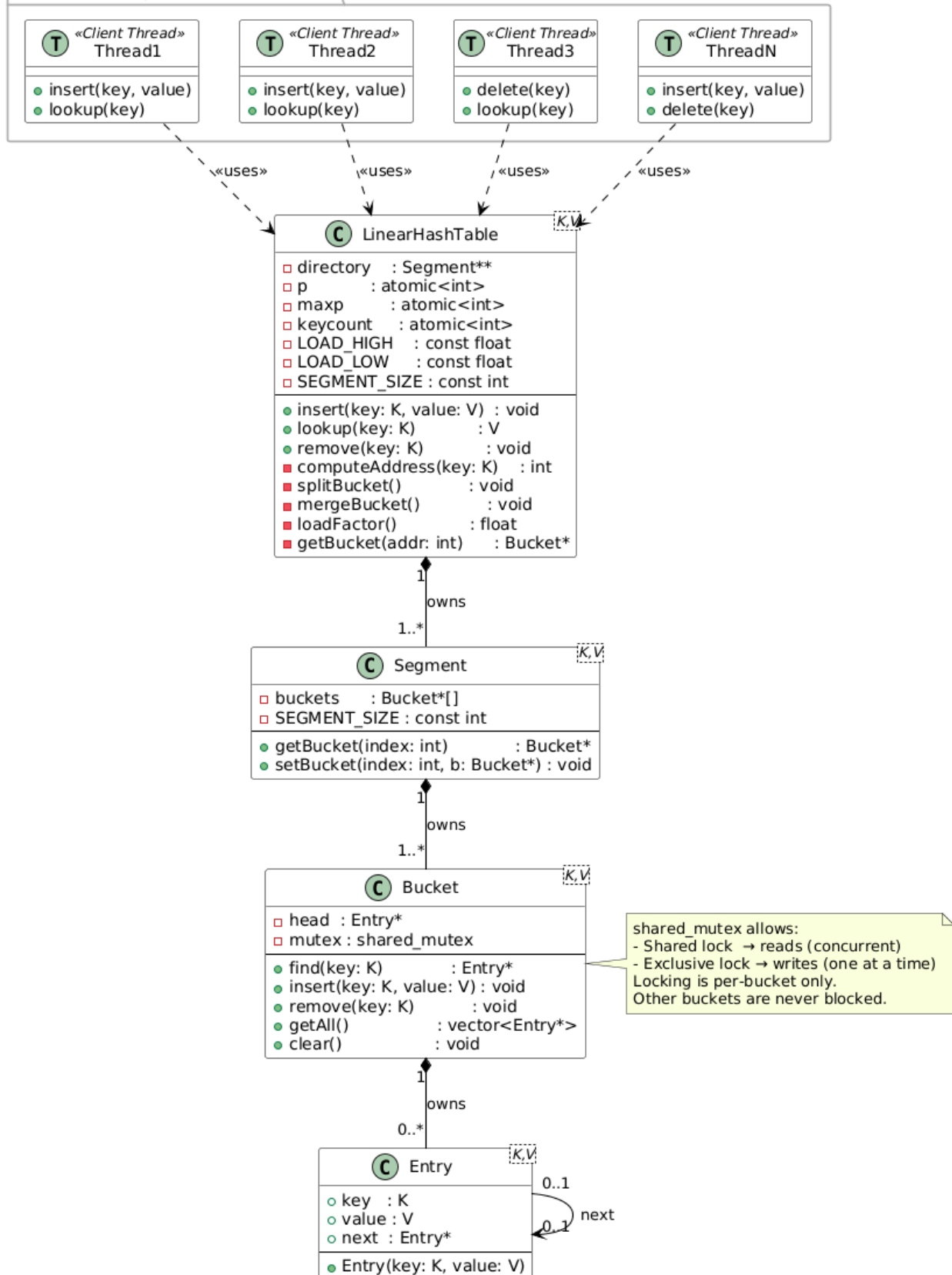
The system is built around four classes arranged in a two-level ownership hierarchy. At the top, `LinearHashTable` holds all the state and logic. Below it, `Segment` groups buckets into fixed-size arrays. Each `Bucket` owns a linked list of records and a mutex for concurrency control. At the bottom, `Entry` represents a single key-value record.

This structure directly reflects Larson's two-level architecture from his 1988 paper — a clean separation between the address computation logic, the storage organisation, and the concurrency mechanism. Each class has one responsibility and one only, which makes the system straightforward to implement, test, and extend with concurrent access

### Architecture Overview:

# SOFTWARE ARCHITECTURE DOCUMENT

## Application Layer — Client Threads



## SOFTWARE ARCHITECTURE DOCUMENT

**Top — Application layer:** Four client threads call insert, lookup, or delete simultaneously. All arrows flow down into a single entry point — the `LinearHashTable`.

**LinearHashTable:** The brain of the system. Holds the three atomic state variables (`p`, `maxp`, `keycount`), the directory pointer, the load thresholds, and all the logic methods. Nothing else in the system knows about splitting or merging.

**Segment:** A dumb fixed-size array of bucket pointers. Its only job is to group buckets together so the directory stays small. No logic lives here.

**Bucket:** The most important class for concurrency. Owns the linked list of records (`head`) and the `std::shared_mutex`. Every lock in the entire system lives inside a `Bucket` — nowhere else.

**Entry:** The simplest class. Just a key, a value, and a `next` pointer. The self-referencing arrow on the right shows how entries chain together into a linked list inside each bucket.

**Left callout:** Reminds you that the `shared_mutex` inside `Bucket` gives two behaviours — shared lock for reads (multiple threads proceed together), exclusive lock for writes (one thread at a time per bucket).

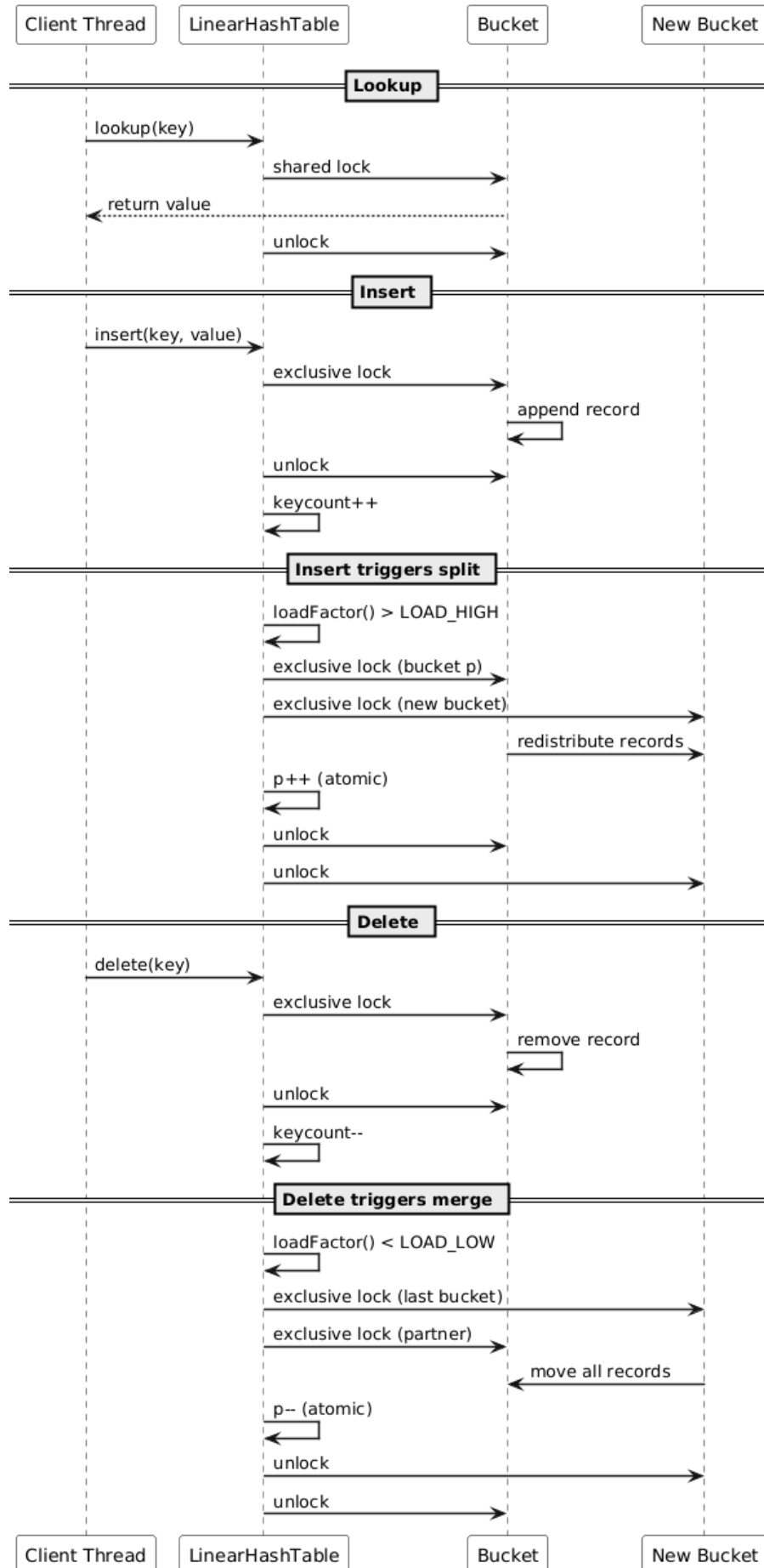
**Right callouts:** One-line responsibility reminder for each class.

## Sequence diagram:

The process view describes how the Concurrent Linear Hash Table behaves at runtime. While the logical view explains what classes exist and how they are structured, the process view explains how those classes come alive when multiple threads are executing simultaneously — how they interact, how they are coordinated, and how conflicts are prevented.

In this system, the central runtime concern is concurrency. Multiple client threads call insert, lookup, and delete at the same time on the same hash table. The process view captures how the per-bucket locking strategy — using `std::shared_mutex` — allows these threads to operate in parallel without corrupting the data. It shows which operations can run concurrently, which must wait, and under what conditions the split worker steps in to grow or shrink the table.

# SOFTWARE ARCHITECTURE DOCUMENT





## SOFTWARE ARCHITECTURE DOCUMENT

Diagram Name: Processes

**Lookup** — shared lock, read, return, unlock

**Insert** — exclusive lock, append, unlock, increment keycount

**Insert triggers split** — load check, lock two buckets, redistribute, p++

**Delete** — exclusive lock, remove, unlock, decrement keycount

**Delete triggers merge** — load check, lock two buckets, move records, p--

### Quality:

1. **Correctness** - is guaranteed by per-bucket `std::shared_mutex` locking and atomic state variables (`p`, `maxp`, `keycount`), ensuring no data races or lost updates under any level of concurrency.
2. **Performance** is strong because shared locks allow fully parallel reads, writes are serialised per bucket only, splits lock exactly two buckets leaving the rest of the table accessible, and no global rehash ever occurs.
3. **Scalability** is built into the design — as the table grows, contention naturally decreases because more buckets means fewer collisions between threads, and the two-level directory expands on demand with no fixed size limit.
4. **Maintainability** is achieved through strict single responsibility per class and by confining all concurrency logic to the `Bucket` class alone, making the system straightforward to understand, test, and extend.
5. **Reliability** is ensured by always acquiring split locks in ascending index order to prevent deadlock, and by the absence of any global lock or global reorganisation, so no single thread can block the entire table

Together these properties make the architecture well suited for an academic implementation of a concurrent dynamic hash table — correct, fast, scalable, and easy to reason about